

microsearch 검색엔진 시험공부식 복습 노트

검색엔진 흐름, 내부 자료구조, BM25 계산, Spring Boot 뉴스 프로젝트에 붙이는 위치까지 한 번에 복습하기

오늘 외울 한 문장

microsearch는 문서를 메모리에 올려 단어별 역색인을 만들고, 검색어가 들어오면 BM25 점수로 URL 또는 문서를 정렬하는 아주 작은 검색엔진이다.

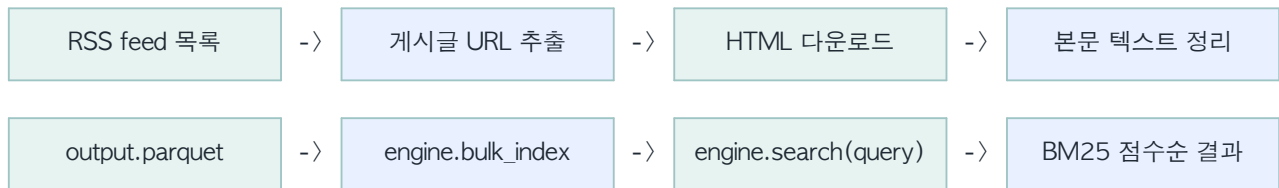
구분	기억할 내용
입력 데이터	RSS에서 모은 글 URL과 본문 텍스트. Spring 프로젝트에서는 일단 News의 title, publisher, authorship을 색인 대상으로 삼을 수 있다.
색인 구조	<code>_index(word)(url) = count</code> 형태. Java로 옮기면 <code>Map<String, Map<Long, Integer>></code> 가 된다.
검색 방식	검색어를 normalize 후 단어로 나누고, 단어마다 BM25 점수를 계산해서 문서별로 합산한다.
결과 정렬	점수가 높은 문서가 더 관련 있는 문서로 간주된다.
Spring 적용 지점	현재 <code>RssService.getNews(keyword, authorship)</code> 의 DB title containing 검색을 <code>SearchEngineService.search(keyword)</code> 로 바꾸는 흐름.

추천 복습법

- 1회독: 큰 흐름 그림만 본다.
- 2회독: `_documents`, `_index`, `bm25`만 손으로 그린다.
- 3회독: Spring의 `RssService.getNews` 위치에 검색엔진을 끼워 넣는다고 상상한다.

1. 전체 흐름 먼저 보기

microsearch의 동작은 크게 두 구간으로 나뉜다. 첫 번째는 검색할 데이터를 모으는 구간이고, 두 번째는 모은 데이터를 검색 가능한 인덱스로 바꾸고 검색 요청에 응답하는 구간이다.



파일	역할	시험식 질문
download_content.py	RSS를 읽고 실제 글 페이지를 받아 본문 텍스트로 정리한 뒤 output.parquet에 저장한다.	검색엔진에 넣을 문서 데이터는 어디서 만들어지나?
app/app.py	FastAPI 앱을 만들고 시작 시 output.parquet를 읽어 SearchEngine에 bulk_index한다.	앱이 켜질 때 색인은 언제 만들어지나?
src/microsearch/engine.py	정규화, 역색인, IDF, BM25, 검색어 점수 합산을 담당한다.	검색의 진짜 핵심 로직은 어디 있나?
Spring RssService	현재 프로젝트에서 뉴스 조회와 keyword 검색 입구 역할을 한다.	Spring에 붙이면 어느 서비스 메서드가 바뀌나?

핵심 감각

크롤러는 검색엔진이 아니다. 크롤러는 문서를 모으고, 검색엔진은 모은 문서를 빠르게 찾기 위한 자료구조와 점수 계산을 가진다.

2. 데이터 수집: RSS에서 검색 대상 만들기

원본 microsearch는 RSS feed를 출발점으로 삼는다. RSS feed 자체는 글 목록과 링크를 담은 XML 문서이고, 실제 검색할 본문은 각 링크의 HTML 페이지에서 가져온다.

순서	함수	하는 일
1	parse_feed(feed_url)	feedparser로 RSS를 읽고 entry.link만 뽑아 게시글 URL 목록을 만든다.
2	fetch_content(session, url)	aiohttp로 게시글 HTML을 비동기 다운로드한다.
3	clean_content(html_content)	BeautifulSoup으로 script/style을 제거하고 화면 텍스트만 이어 붙인다.
4	process_feed(feed_url, session, loop)	한 feed의 모든 글 URL과 본문을 [(url, content), ...] 형태로 묶는다.
5	main(feed_file)	여러 feed 결과를 평평한 리스트로 만들고 output.parquet로 저장한다.

왜 비동기를 썼을까?

웹 페이지 여러 개를 받는 작업은 대부분 네트워크 응답 대기 시간이다. 하나씩 기다리면 느리지만, aiohttp와 asyncio.gather를 쓰면 여러 요청을 동시에 보내고 응답이 오는 대로 모을 수 있다.

```
tasks = [fetch_content(session, post_url) for post_url in post_urls]
post_contents = await asyncio.gather(*tasks)
cleaned_contents = [clean_content(content) for content in post_contents]
```

Spring 프로젝트와의 차이

현재 Spring 프로젝트는 ROME으로 RSS 항목을 파싱해 News 엔티티에 title, url, publisher, publishedAt, authorship을 저장한다. 원본 microsearch처럼 게시글 본문 HTML까지 저장하지는 않는다. 그래서 첫 적용은 title 중심 검색이 가장 자연스럽다.

3. 앱 시작: parquet를 읽고 메모리 인덱스 만들기

원본 FastAPI 앱은 서버가 시작될 때 output.parquet를 읽는다. 그 다음 URL과 content를 묶어 SearchEngine.bulk_index에 넣는다. 이 순간부터 검색 요청이 가능해진다.

```
data = pd.read_parquet(args.data_path)
content = list(zip(data["URL"].values, data["content"].values))
engine.bulk_index(content)
run(app, host="127.0.0.1", port=8000)
```

단계	메모리 상태
engine = SearchEngine()	_documents와 _index가 비어 있다.
bulk_index(content)	문서 하나씩 index(url, content)를 호출한다.
index 실행 후	_documents에는 원문, _index에는 단어별 등장 횟수가 쌓인다.
검색 요청 시	DB나 파일을 다시 읽는 것이 아니라 이미 만들어진 메모리 인덱스를 사용한다.

Spring으로 옮길 때

Spring에서는 애플리케이션 시작 후 DB에 저장된 News를 전부 읽어 SearchEngineService.bulkIndex(newsList)를 호출하거나, 첫 검색 시 lazy하게 색인을 만들 수 있다. 수집 후 새 뉴스가 저장되면 그 뉴스만 추가 색인하면 된다.

4. 핵심 자료구조: _documents와 _index

검색엔진의 기억 장치는 두 개라고 보면 된다. _documents는 원본 문서 저장소이고, _index는 검색을 빠르게 하기 위한 역색인이다.

이름	Python 원본	Java/Spring 포팅 시
원본 문서	_documents: dict(str, str) URL -> content	Map<Long, String> documents newsId -> searchableText
역색인	_index: dict(str, dict(str, int)) word -> URL -> count	Map<String, Map<Long, Integer>> index word -> newsId -> count
문서 목록	posts property가 _documents.keys() 반환	documents.keySet() 또는 DB 조회 결과

작은 예제로 손으로 그리기

문서 3개를 색인한다고 생각해 보자.

```
doc1: "spring boot search"
doc2: "spring rss news"
doc3: "python search engine"

_documents:
doc1 -> "spring boot search"
doc2 -> "spring rss news"
doc3 -> "python search engine"

_index:
spring -> doc1: 1, doc2: 1
search -> doc1: 1, doc3: 1
boot -> doc1: 1
rss -> doc2: 1
news -> doc2: 1
python -> doc3: 1
engine -> doc3: 1
```

역색인의 의미

검색어가 spring이면 모든 문서를 훑지 않고 _index['spring']만 보면 된다. 그러면 spring이 들어간 문서와 등장 횟수를 바로 알 수 있다.

5. normalize_string: 검색 전 텍스트 정리

검색엔진은 같은 의미의 텍스트가 가능한 한 같은 단어로 들어오길 원한다. 그래서 색인할 때도, 검색할 때도 같은 normalize_string을 통과시킨다.

처리	예시
문장부호 제거	Hello, world! -> Hello world
여러 공백 정리	Hello world -> Hello world
소문자 변환	Hello World -> hello world
공백 기준 분리	hello world -> ['hello', 'world']

```
def normalize_string(input_string):  
    translation_table = str.maketrans(string.punctuation, " " * len(string.punctuation))  
    string_without_punc = input_string.translate(translation_table)  
    string_without_double_spaces = " ".join(string_without_punc.split())  
    return string_without_double_spaces.lower()
```

한국어 검색 주의

한국어는 영어처럼 공백만으로 단어가 잘 나뉘지 않는 경우가 많다. 예를 들어 '스프링부트'와 '스프링 부트'는 다르게 처리될 수 있다. 처음에는 제목 검색으로 시작하고, 나중에 형태소 분석기나 n-gram 방식을 검토하면 된다.

6. index와 bulk_index: 문서를 인덱스에 넣는 순간

SearchEngine.index(url, content)는 한 문서를 등록한다. bulk_index는 이 index를 여러 번 호출하는 얇은 반복문이다.

```
def index(self, url, content):
    self._documents[url] = content
    words = normalize_string(content).split(" ")
    for word in words:
        self._index[word][url] += 1
    if hasattr(self, "_avdl"):
        del self._avdl
```

라인의 의미	설명
self._documents[url] = content	나중에 문서 길이 계산과 문서 목록 확인을 위해 원문을 저장한다.
normalize_string(content).split(' ')	문장부호, 대소문자, 공백을 정리한 뒤 단어 배열로 만든다.
self._index[word][url] += 1	이 단어가 이 문서에 몇 번 등장했는지 센다.
del self._avdl	문서가 추가되면 평균 문서 길이가 바뀌므로 캐시를 무효화한다.

defaultdict 감각

Python의 defaultdict(lambda: defaultdict(int)) 덕분에 처음 보는 word나 url도 바로 += 1 할 수 있다. Java에서는 computeIfAbsent를 쓰면 비슷한 모양이 된다.

```
index.computeIfAbsent(word, key -> new HashMap<>())
    .merge(newslid, 1, Integer::sum);
```

7. BM25: 왜 단순 포함 검색보다 나은가

DB의 title containing 검색은 '들어 있냐, 아니냐'에 가깝다. BM25는 들어 있는 문서들 사이에서도 어떤 문서가 더 관련 있는지 점수를 계산한다.

요소	질문	점수에 미치는 영향
freq	이 단어가 문서에 몇 번 나오나?	자주 나오면 점수가 올라간다. 다만 무한정 커지지는 않도록 완만해진다.
idf	이 단어가 전체 문서 중 얼마나 희귀한가?	희귀한 단어일수록 점수가 크다.
dl	문서 길이가 얼마나 긴가?	긴 문서는 단어가 우연히 많이 나올 수 있으므로 보정한다.
avdl	전체 평균 문서 길이는 얼마인가?	문서 길이 보정의 기준이 된다.
k1, b	빈도와 길이 보정을 얼마나 강하게 할까?	일반적으로 k1=1.5, b=0.75를 많이 쓴다.

원본 공식 느낌

$$\text{idf} = \log((N - n_{\text{kw}} + 0.5) / (n_{\text{kw}} + 0.5) + 1)$$

$$\text{score} = \text{idf} * \text{freq} * (k1 + 1) / (\text{freq} + k1 * (1 - b + b * dl / \text{avdl}))$$

시험식 암기

BM25는 '희귀한 단어가, 해당 문서에 충분히 나오고, 문서 길이까지 고려했을 때 얼마나 중요한가'를 계산한다.

8. search(query): 여러 단어 점수를 합산한다

검색어가 한 단어라면 bm25 한 번이면 된다. 검색어가 여러 단어라면 각 단어의 점수를 문서별로 더한다.

```
def search(self, query):
    keywords = normalize_string(query).split(" ")
    url_scores = {}
    for kw in keywords:
        kw_urls_score = self.bm25(kw)
        url_scores = update_url_scores(url_scores, kw_urls_score)
    return url_scores
```

검색어	내부 처리
spring search	('spring', 'search')로 분리한다.
spring	spring이 들어간 문서들의 BM25 점수를 구한다.
search	search가 들어간 문서들의 BM25 점수를 구한다.
합산	같은 문서가 양쪽에 있으면 점수를 더한다.
정렬	app.py의 get_top_urls가 점수 내림차순으로 상위 N개를 뽑는다.

손계산 흐름

doc1='spring boot search', doc2='spring rss news', doc3='python search engine'에서 'spring search'를 검색한다고 하자.

- spring 후보: doc1, doc2
- search 후보: doc1, doc3
- doc1은 spring과 search 둘 다 맞으므로 두 점수를 더 받는다.
- doc2는 spring 점수만 받는다.
- doc3은 search 점수만 받는다.
- 최종적으로 doc1이 위로 올라올 가능성이 높다.

9. 현재 Spring 프로젝트에서 검색 흐름

현재 Spring 뉴스 프로젝트는 이미 keyword 검색 입구가 있다. 다만 지금은 검색엔진이 아니라 DB 메서드 이름 쿼리로 title containing 검색을 한다.

/news?keyword=...

->

RssController.newsList

->

RssService.getNews

->

NewsRepository

```
public List<NewsResponse> getNews(String keyword, String authorship) {
    if (hasKeyword && hasAuthorship) {
        newsList = repository
            .findByTitleContainingAndAuthorshipOrderByPublishedAtDesc(keyword, authorship);
    } else if (hasKeyword) {
        newsList = repository.findByTitleContainingOrderByPublishedAtDesc(keyword);
    }
    ...
}
```

현재 방식	microsearch 방식
DB가 title에 keyword가 포함된 행을 찾는다.	메모리 인덱스가 keyword가 들어간 문서 후보를 바로 찾는다.
정렬은 publishedAt desc 중심이다.	정렬은 검색 점수 desc 중심이 된다.
검색 대상은 title이다.	title, publisher, authorship, 나중에는 본문까지 합쳐 색인할 수 있다.

바꿀 위치

가장 먼저 바꿀 곳은 RssService.getNews(keyword, authorship)이다. keyword가 있으면 Repository의 containing 검색 대신 SearchEngineService.search(keyword)를 호출하고, 그 결과 newsId 순서대로 NewsResponse를 만들면 된다.

10. Spring에 붙이는 추천 구조

처음부터 본문 크롤링까지 욕심내기보다, 현재 DB에 있는 News 데이터로 검색엔진을 붙이는 것이 좋다. 색인 대상 문자열은 title, publisher, authorship을 합친다.

클래스	책임
SearchEngineService	News를 색인하고 keyword 검색 시 newsId와 score를 반환한다.
SearchResult	newsId, score를 담은 작은 DTO 또는 record.
RssService	keyword가 있으면 SearchEngineService를 호출하고, authorship 필터와 NewsResponse 변환을 담당한다.
NewsRepository	검색 결과 id 목록에 해당하는 News를 가져온다. 순서는 검색 점수 순서를 유지해야 한다.
collectNews 이후	새로 저장된 News를 SearchEngineService에도 추가 색인한다.

Java 포팅 의사코드

```
class SearchEngineService {
    private Map<String, Map<Long, Integer>> index = new HashMap<>();
    private Map<Long, String> documents = new HashMap<>();

    void index(News news) {
        String text = news.getTitle() + " " + news.getPublisher() + " " + news.getAuthorship();
        documents.put(news.getId(), text);
        for (String word : normalize(text)) {
            index.computeIfAbsent(word, k -> new HashMap<>())
                .merge(news.getId(), 1, Integer::sum);
        }
    }

    List<SearchResult> search(String query) {
        // normalize -> bm25 per keyword -> score merge -> sort desc
    }
}
```

순서 유지 주의

DB에서 findAllById(ids)를 호출하면 검색 점수 순서가 보장되지 않을 수 있다. search 결과의 id 순서를 기준으로 다시 정렬하거나, Map<Long, News>로 바꾼 뒤 SearchResult 순서대로 NewsResponse를 만들어야 한다.

11. 구현할 때 자주 걸리는 부분

문제	원인	해결 감각
검색 결과가 비어 있음	색인이 아직 만들어지지 않았거나 normalize 방식이 색인/검색에서 다름	앱 시작 시 bulkIndex가 실행되는지, 같은 normalize를 쓰는지 확인한다.
새 뉴스가 검색되지 않음	DB 저장 후 검색엔진에 추가 색인을 안 함	collectNews에서 saveAll 후 저장된 News를 index한다.
점수순이 아니라 날짜 순으로 나옴	검색 후 Repository 정렬 메서드를 다시 타서 순서가 바뀜	SearchResult 순서를 끝까지 유지한다.
한글 검색 품질이 낮음	공백 기준 토큰화의 한계	처음엔 제목 검색으로 만족하고, 이후 n-gram 또는 형태소 분석을 검토한다.
동시성 문제	수집 스케줄러와 검색 요청이 동시에 index를 읽고 씀	초기 버전은 synchronized로 단순화하고, 이후 ConcurrentHashMap 또는 재빌드 교체 방식을 쓴다.
avgd 계산 오류	문서가 0개일 때 평균 길이 계산	문서가 없으면 검색 결과 빈 리스트를 반환한다.

가장 작은 1차 목표

- SearchEngineService를 만든다.
- 앱 시작 시 repository.findAll()로 기존 News를 bulkIndex한다.
- collectNews가 새 뉴스를 저장한 뒤 SearchEngineService.index(news)를 호출한다.
- /news?keyword=java 요청에서 SearchEngineService.search('java')를 사용한다.
- 검색 결과는 BM25 점수순, keyword가 없을 때는 기존처럼 publishedAt desc를 유지한다.

12. 시험 전 체크 문제

Q1. `_index{word}{url} = count`에서 `count`는 무엇인가?

그 단어가 해당 문서에 등장한 횟수다. BM25의 `freq`로 쓰인다.

Q2. 왜 검색할 때 전체 문서를 전부 훑지 않아도 되는가?

역색인에서 단어를 키로 바로 후보 문서 목록을 찾기 때문이다.

Q3. `idf`가 큰 단어는 어떤 단어인가?

전체 문서 중 적은 문서에만 등장하는 희귀한 단어다.

Q4. 문서를 새로 추가하면 왜 `avgdl` 캐시를 지우는가?

평균 문서 길이가 바뀌기 때문이다. 이전 평균으로 점수를 계산하면 길이 보정이 들어진다.

Q5. Spring 프로젝트에서 현재 keyword 검색은 어디서 처리되는가?

`RssController.newsList`가 keyword를 받고, `RssService.getNews`가 `Repository`의 `title containing` 메서드를 호출한다.

Q6. microsearch 방식으로 바꾸면 결과 정렬 기준은 무엇이 되는가?

`publishedAt desc`가 아니라 `BM25 score desc`가 된다. keyword가 없을 때만 날짜순을 유지하면 자연스럽다.

Q7. Java로 옮길 때 URL 대신 `newsId`를 쓰는 이유는?

`News` 엔티티의 기본키가 있고, 북마크와 화면 렌더링도 `id` 중심으로 연결되어 있어서 DB 재조회와 정렬 관리가 쉽다.

13. 마지막 30초 요약

키워드	한 줄 암기
크롤링	검색할 문서를 모으는 단계다.
정규화	색인과 검색이 같은 기준으로 단어를 보게 만드는 단계다.
역색인	word -> document -> count 구조로 후보 문서를 빠르게 찾는 자료구조다.
BM25	희귀성, 등장 빈도, 문서 길이를 함께 고려해 검색 관련도를 점수화한다.
Spring 적용	RssService.getNews의 keyword 분기를 SearchEngineService 검색으로 바꾸면 된다.

최종 그림

Spring이 RSS로 News를 모은다. SearchEngineService가 News를 색인한다. 사용자가 keyword를 입력한다. 검색 엔진이 BM25 점수순 newsId를 반환한다. RssService가 그 순서대로 NewsResponse를 만들어 화면에 보여준다.

참고한 로컬 파일

- /Users/yoo/Documents/microsearch-lab/microsearch/download_content.py
- /Users/yoo/Documents/microsearch-lab/microsearch/app/app.py
- /Users/yoo/Documents/microsearch-lab/microsearch/src/microsearch/engine.py
- /Users/yoo/Desktop/springboot-developer/src/main/java/me/shinsunyoung/springbootdeveloper/service/RssService.java
- /Users/yoo/Desktop/springboot-developer/src/main/java/me/shinsunyoung/springbootdeveloper/controller/RssController.java